> 📚 참고: 이 실습은 『파이썬 라이브러리를 활용한 머신러닝』(안드레아스 뮐러·세라 가이도 지음, 박해선 옮김, 한빛미디어) 예제와 저자의 `mglearn` 패키지(BSD 라이선스)를 바탕으로 합니다.

> 🛠️ **2026.10 수정본 (v11)** — 원본: `ch02_01_04_binary_classification.ipynb`
>
> - `LogisticRegression(penalty="none")` 문자열이 더는 안 받아져서 `penalty=None`으로 바꿨어요.
> - 윈도우 폰트 경로 대신 코랩·맥·윈도우 공통 한글 폰트 설정(koreanize-matplotlib)을 씁니다.

분류형 선형 모델

학습 내용

- 이진 분류의 알고리즘에 대해 알아봅니다.
- 심플한 데이터를 활용하여 이진 분류 알고리즘을 실습해 봅니다.
- 로지스틱 회귀 모델과 서포트벡터 머신에 대해 알아보며, 규제 매개변수 C에 대해 알아봅니다.

- 분류용 선형 모델에서는 **결정경계**가 입력의 선형 함수
- 이진 선형 분류기는 선, 평면, 초평면을 사용해서 **두 개의 클래스를 구분하는 분류기**

이진 분류를 위한 예측 방정식

- y = w[0] * x[0] + w[1] * x[1] + w[2] * x[2] + ... + w[p] * x[p] + b > 0

- 특성들의 가중치 합을 사용하는 대신, 예측한 값을 임계치 0과 비교.
- **0보다 작으면 클래스를 -1, 0보다 크면 1**로 예측

이진 분류를 위한 잘 알려진 알고리즘

- 로지스틱 회귀(Logistic regression)
- 서포트 벡터 머신(support vector machine)

In [1]:
```
# 2026.10 수정: 윈도우 폰트 경로 대신 공통 한글 폰트 설정
import matplotlib
import matplotlib.pyplot as plt
%matplotlib inline
```

```
# 한글 폰트 설정 (코랩·윈도우·맥 공통)
%pip install -q koreanize-matplotlib setuptools
import setuptools  # 2026.10 수정: Python 3.12+엔 distutils가 없어 koreanize-ma
import koreanize_matplotlib  # 나눔고딕을 matplotlib 기본 폰트로 등록
```

```
 3.11.9

Note: See 'pyenv help global' for tips on allowing multiple
      Python versions to be found at the same time.
```

라이브러리 불러오기

In [2]:
```
from sklearn.linear_model import LogisticRegression
from sklearn.svm import LinearSVC
import mglearn
import matplotlib.pyplot as plt
```

데이터셋 로드

- 26개의 행과 2개의 열의 입력
- 26개의 출력

In [3]:
```
X, y = mglearn.datasets.make_forge()
print(X.shape, y.shape)
print(X[:5], y[:5])  # 5개 데이터
```

```
(26, 2) (26,)
[[ 9.96346605  4.59676542]
 [11.0329545  -0.16816717]
 [11.54155807  5.21116083]
 [ 8.69289001  1.54322016]
 [ 8.1062269   4.28695977]] [1 0 1 0 0]
```

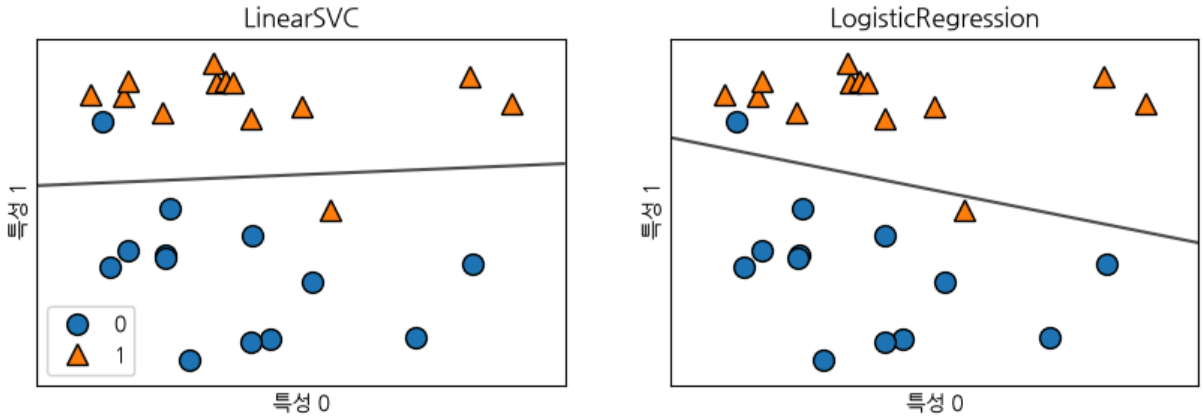
데이터 셋을 이용하여 로지스틱 회귀와 서포트벡터 머신 구현

- LogisticRegression과 LinearSVC모델 생성
- 데이터 셋 : forge 데이터 셋
- 모델 : LogisticRegression과 LinearSVC모델

In [4]:
```
fig, axes = plt.subplots(1, 2, figsize=(10,3))

for model, ax in zip([LinearSVC(), LogisticRegression()], axes):
    clf = model.fit(X, y)
    mglearn.plots.plot_2d_separator(clf, X, fill=False, eps=0.5, ax=ax, alph
    mglearn.discrete_scatter(X[:,0], X[:,1], y, ax=ax)
    ax.set_title(clf. __class__.__name__)
    ax.set_xlabel("특성 0")
    ax.set_ylabel("특성 1")
axes[0].legend()
```

Out[4]:
```
<matplotlib.legend.Legend at 0x1161bbec0>
```



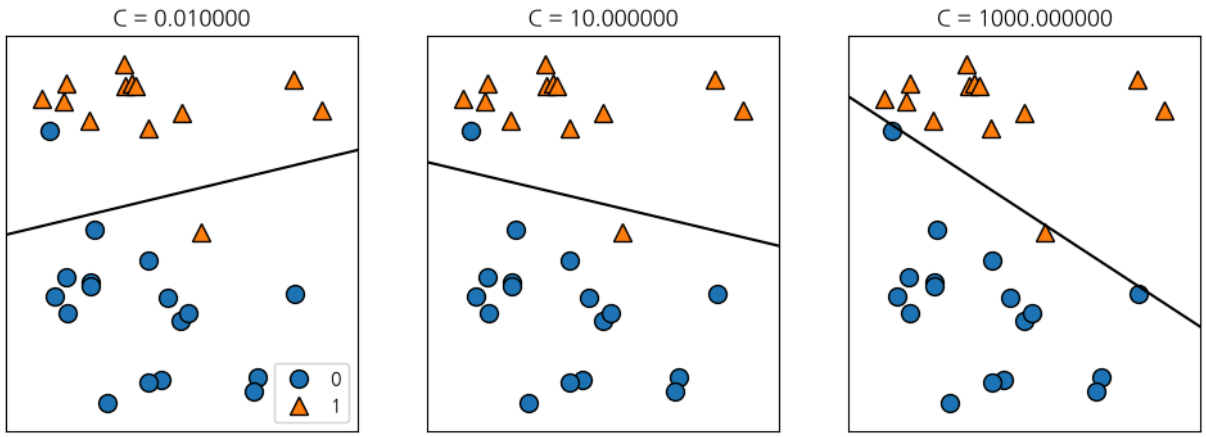
- 위쪽은 클래스 1, 아래쪽은 클래스 0으로 분류.
  - 새로운 데이터가 **위쪽에 놓이면 클래스 1로 분류**
  - 새로운 데이터가 반대로 **직선 아래쪽에 놓이면 클래스 0으로 분류**
- 두개의 모델은 기본적으로 L2규제(계수를 0에 가깝게)를 이용

매개변수 C

- LogisticRegression과 LinearSVC에서 규제의 강도를 결정하는 매개변수 C
  - C의 값이 높아지면 규제가 감소
    - 높은 C의 값을 지정하면 LogisticRegression과 LinearSVC 학습용 데이터 세트에 최대한 맞추게 됨.
  - C의 값이 낮추면 모델 계수 벡터(w)가 0에 가까워지도록 함.

각기 다른 C값으로 만든 SVM 모델의 결정 경계

In [5]:
```
mglearn.plots.plot_linear_svc_regularization()
```



- 왼쪽 그림은 아주 작은 C값으로 인해 규제가 많이 적용됨.
- 규제가 비교적 강해진 모델은 비교적 수평에 가까운 결정
- C값을 아주 크게한 오른쪽은 마침내 클래스의 0의 모든 데이터를 올바르게 분류함.

유방암 데이터셋을 활용한 LogisticRegression

In [6]:
```
from sklearn.datasets import load_breast_cancer
cancer = load_breast_cancer()
from sklearn.model_selection import train_test_split
```

In [7]:
```
X_train, X_test, y_train, y_test = train_test_split(cancer.data, cancer.targ
                                                    stratify=cancer.target,
                                                    random_state=42)
# C의 기본값은 1
logreg = LogisticRegression(C=1).fit(X_train, y_train)

print("훈련 세트 점수 : {:.3f}".format(logreg.score(X_train, y_train)))
print("테스트 세트 점수 : {:.3f}".format(logreg.score(X_test, y_test)))
```

```
훈련 세트 점수 : 0.946
테스트 세트 점수 : 0.958
.../site-packages/sklearn/linear_model/_logistic.py:599: ConvergenceWarning:
lbfgs failed to converge after 100 iteration(s) (status=1):
STOP: TOTAL NO. OF ITERATIONS REACHED LIMIT

Increase the number of iterations to improve the convergence (max_iter=100).
You might also want to scale the data as shown in:
    https://scikit-learn.org/stable/modules/preprocessing.html
Please also refer to the documentation for alternative solver options:
    https://scikit-learn.org/stable/modules/linear_model.html#logistic-regre
ssion
  n_iter_i = _check_optimize_result(
```

- 기본값 C=1이 훈련세트와 테스트 양쪽이 95%로 정확도로 꽤 훌륭함.
  - 단, 훈련세트와 테스트 세트의 성능이 매우 비슷하므로 과소적합으로 판단됨.
    - 모델의 제약을 풀기 위해 C를 증가시켜봄.

In [8]:
```
logreg100 = LogisticRegression(C=100).fit(X_train, y_train)

print("훈련 세트 점수 : {:.3f}".format(logreg100.score(X_train, y_train)))
print("테스트 세트 점수 : {:.3f}".format(logreg100.score(X_test, y_test)))
```

```
훈련 세트 점수 : 0.946
테스트 세트 점수 : 0.958
.../site-packages/sklearn/linear_model/_logistic.py:599: ConvergenceWarning:
lbfgs failed to converge after 100 iteration(s) (status=1):
STOP: TOTAL NO. OF ITERATIONS REACHED LIMIT

Increase the number of iterations to improve the convergence (max_iter=100).
You might also want to scale the data as shown in:
    https://scikit-learn.org/stable/modules/preprocessing.html
Please also refer to the documentation for alternative solver options:
    https://scikit-learn.org/stable/modules/linear_model.html#logistic-regre
ssion
  n_iter_i = _check_optimize_result(
```

- 훈련 세트 점수가 증가 -> 복잡도가 높은 모델일 수록 성능이 좋음.
  - C의 값이 증가되면 복잡도가 증가
  - C의 값이 감소되면 제약이 많아 단순해짐

In [9]:
```
logreg001 = LogisticRegression(C=0.01).fit(X_train, y_train)

print("학습용 세트 점수 : {:.3f}".format(logreg001.score(X_train, y_train)))
print("테스트 세트 점수 : {:.3f}".format(logreg001.score(X_test, y_test)))
```

```
학습용 세트 점수 : 0.934
테스트 세트 점수 : 0.930
```

```
.../site-packages/sklearn/linear_model/_logistic.py:599: ConvergenceWarning:
lbfgs failed to converge after 100 iteration(s) (status=1):
STOP: TOTAL NO. OF ITERATIONS REACHED LIMIT

Increase the number of iterations to improve the convergence (max_iter=100).
You might also want to scale the data as shown in:
    https://scikit-learn.org/stable/modules/preprocessing.html
Please also refer to the documentation for alternative solver options:
    https://scikit-learn.org/stable/modules/linear_model.html#logistic-regre
ssion
  n_iter_i = _check_optimize_result(
```

실습 - 매개변수 C의 값 변경

- C의 값을 변경해 가면서 0.0001, 0.001, 0.01, 0.1, 1, 1에 대하여 정확도를 확인해 보자.

In [10]:
```
import numpy as np
```

In [11]:
```
c_param = [0.0001, 0.001, 0.01, 0.1, 1, 1]

for p in c_param:
    logreg = LogisticRegression(C=0.01).fit(X_train, y_train)
    lr_tr = logreg.score(X_train, y_train)
    lr_test = logreg.score(X_test, y_test)
    print("C : {} 학습:{:.3f} 테스트:{:.3f}".format(p, lr_tr, lr_test))
    print("유효한 feature 개수 : ", np.sum(logreg.coef_ !=0))
```

```
C : 0.0001 학습:0.934 테스트:0.930
유효한 feature 개수 :  30
C : 0.001 학습:0.934 테스트:0.930
유효한 feature 개수 :  30
C : 0.01 학습:0.934 테스트:0.930
유효한 feature 개수 :  30
C : 0.1 학습:0.934 테스트:0.930
유효한 feature 개수 :  30
C : 1 학습:0.934 테스트:0.930
유효한 feature 개수 :  30
C : 1 학습:0.934 테스트:0.930
유효한 feature 개수 :  30
```

```
.../site-packages/sklearn/linear_model/_logistic.py:599: ConvergenceWarning:
lbfgs failed to converge after 100 iteration(s) (status=1):
STOP: TOTAL NO. OF ITERATIONS REACHED LIMIT

Increase the number of iterations to improve the convergence (max_iter=100).
You might also want to scale the data as shown in:
    https://scikit-learn.org/stable/modules/preprocessing.html
Please also refer to the documentation for alternative solver options:
    https://scikit-learn.org/stable/modules/linear_model.html#logistic-regre
ssion
  n_iter_i = _check_optimize_result(
.../site-packages/sklearn/linear_model/_logistic.py:599: ConvergenceWarning:
lbfgs failed to converge after 100 iteration(s) (status=1):
STOP: TOTAL NO. OF ITERATIONS REACHED LIMIT

Increase the number of iterations to improve the convergence (max_iter=100).
You might also want to scale the data as shown in:
    https://scikit-learn.org/stable/modules/preprocessing.html
Please also refer to the documentation for alternative solver options:
    https://scikit-learn.org/stable/modules/linear_model.html#logistic-regre
ssion
  n_iter_i = _check_optimize_result(
.../site-packages/sklearn/linear_model/_logistic.py:599: ConvergenceWarning:
lbfgs failed to converge after 100 iteration(s) (status=1):
STOP: TOTAL NO. OF ITERATIONS REACHED LIMIT

Increase the number of iterations to improve the convergence (max_iter=100).
You might also want to scale the data as shown in:
    https://scikit-learn.org/stable/modules/preprocessing.html
Please also refer to the documentation for alternative solver options:
    https://scikit-learn.org/stable/modules/linear_model.html#logistic-regre
ssion
  n_iter_i = _check_optimize_result(
.../site-packages/sklearn/linear_model/_logistic.py:599: ConvergenceWarning:
lbfgs failed to converge after 100 iteration(s) (status=1):
STOP: TOTAL NO. OF ITERATIONS REACHED LIMIT

Increase the number of iterations to improve the convergence (max_iter=100).
You might also want to scale the data as shown in:
    https://scikit-learn.org/stable/modules/preprocessing.html
Please also refer to the documentation for alternative solver options:
    https://scikit-learn.org/stable/modules/linear_model.html#logistic-regre
ssion
  n_iter_i = _check_optimize_result(
.../site-packages/sklearn/linear_model/_logistic.py:599: ConvergenceWarning:
lbfgs failed to converge after 100 iteration(s) (status=1):
STOP: TOTAL NO. OF ITERATIONS REACHED LIMIT

Increase the number of iterations to improve the convergence (max_iter=100).
You might also want to scale the data as shown in:
    https://scikit-learn.org/stable/modules/preprocessing.html
Please also refer to the documentation for alternative solver options:
    https://scikit-learn.org/stable/modules/linear_model.html#logistic-regre
ssion
  n_iter_i = _check_optimize_result(
.../site-packages/sklearn/linear_model/_logistic.py:599: ConvergenceWarning:
```

```
lbfgs failed to converge after 100 iteration(s) (status=1):
STOP: TOTAL NO. OF ITERATIONS REACHED LIMIT

Increase the number of iterations to improve the convergence (max_iter=100).
You might also want to scale the data as shown in:
    https://scikit-learn.org/stable/modules/preprocessing.html
Please also refer to the documentation for alternative solver options:
    https://scikit-learn.org/stable/modules/linear_model.html#logistic-regre
ssion
  n_iter_i = _check_optimize_result(
```

In [12]:

```
logreg100 = LogisticRegression(C=100).fit(X_train, y_train)
logreg = LogisticRegression(C=1).fit(X_train, y_train)
logreg001 = LogisticRegression(C=0.01).fit(X_train, y_train)

plt.plot(logreg100.coef_.T, '^', label="C=100")
plt.plot(logreg.coef_.T, 'o', label="C=1")
plt.plot(logreg001.coef_.T, 'v', label="C=0.01")

xlims = plt.xlim()
plt.hlines(0, xlims[0], xlims[1])

plt.ylim(-5, 5)
plt.xlabel("특성")
plt.ylabel("계수 크기")
plt.legend()
```

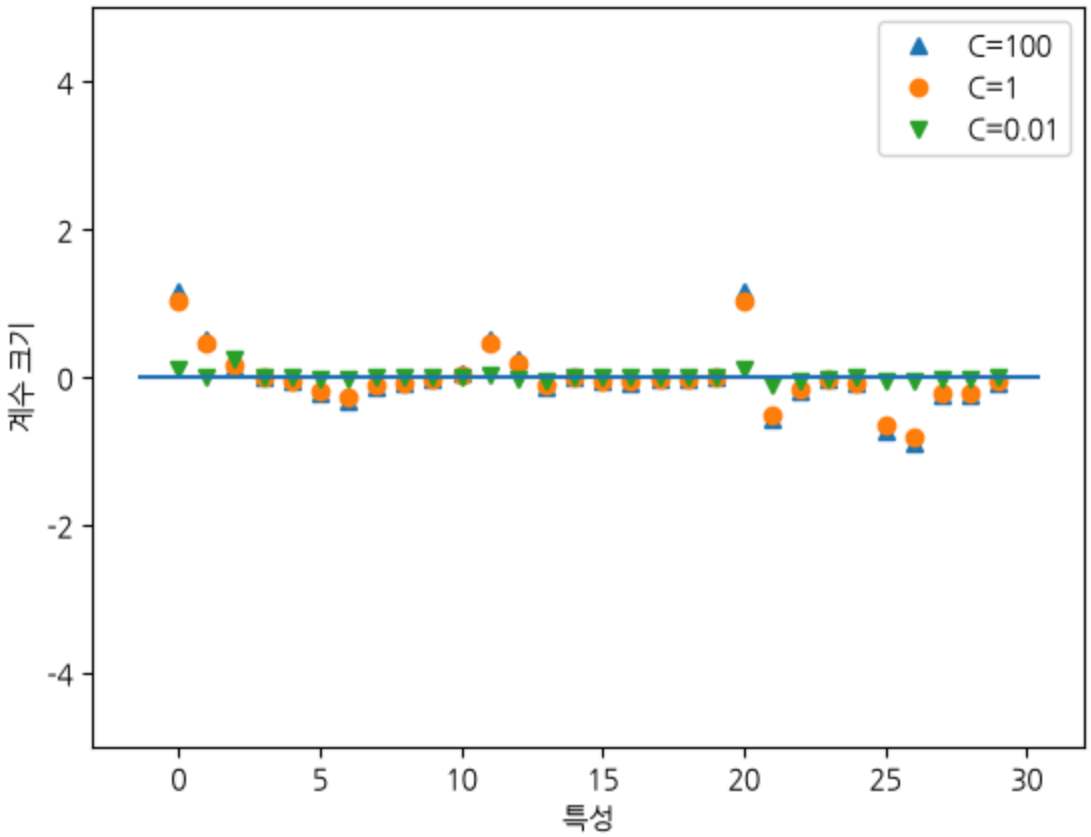
```
.../site-packages/sklearn/linear_model/_logistic.py:599: ConvergenceWarning:
lbfgs failed to converge after 100 iteration(s) (status=1):
STOP: TOTAL NO. OF ITERATIONS REACHED LIMIT

Increase the number of iterations to improve the convergence (max_iter=100).
You might also want to scale the data as shown in:
    https://scikit-learn.org/stable/modules/preprocessing.html
Please also refer to the documentation for alternative solver options:
    https://scikit-learn.org/stable/modules/linear_model.html#logistic-regre
ssion
  n_iter_i = _check_optimize_result(
.../site-packages/sklearn/linear_model/_logistic.py:599: ConvergenceWarning:
lbfgs failed to converge after 100 iteration(s) (status=1):
STOP: TOTAL NO. OF ITERATIONS REACHED LIMIT

Increase the number of iterations to improve the convergence (max_iter=100).
You might also want to scale the data as shown in:
    https://scikit-learn.org/stable/modules/preprocessing.html
Please also refer to the documentation for alternative solver options:
    https://scikit-learn.org/stable/modules/linear_model.html#logistic-regre
ssion
  n_iter_i = _check_optimize_result(
.../site-packages/sklearn/linear_model/_logistic.py:599: ConvergenceWarning:
lbfgs failed to converge after 100 iteration(s) (status=1):
STOP: TOTAL NO. OF ITERATIONS REACHED LIMIT

Increase the number of iterations to improve the convergence (max_iter=100).
You might also want to scale the data as shown in:
    https://scikit-learn.org/stable/modules/preprocessing.html
Please also refer to the documentation for alternative solver options:
    https://scikit-learn.org/stable/modules/linear_model.html#logistic-regre
ssion
  n_iter_i = _check_optimize_result(
```

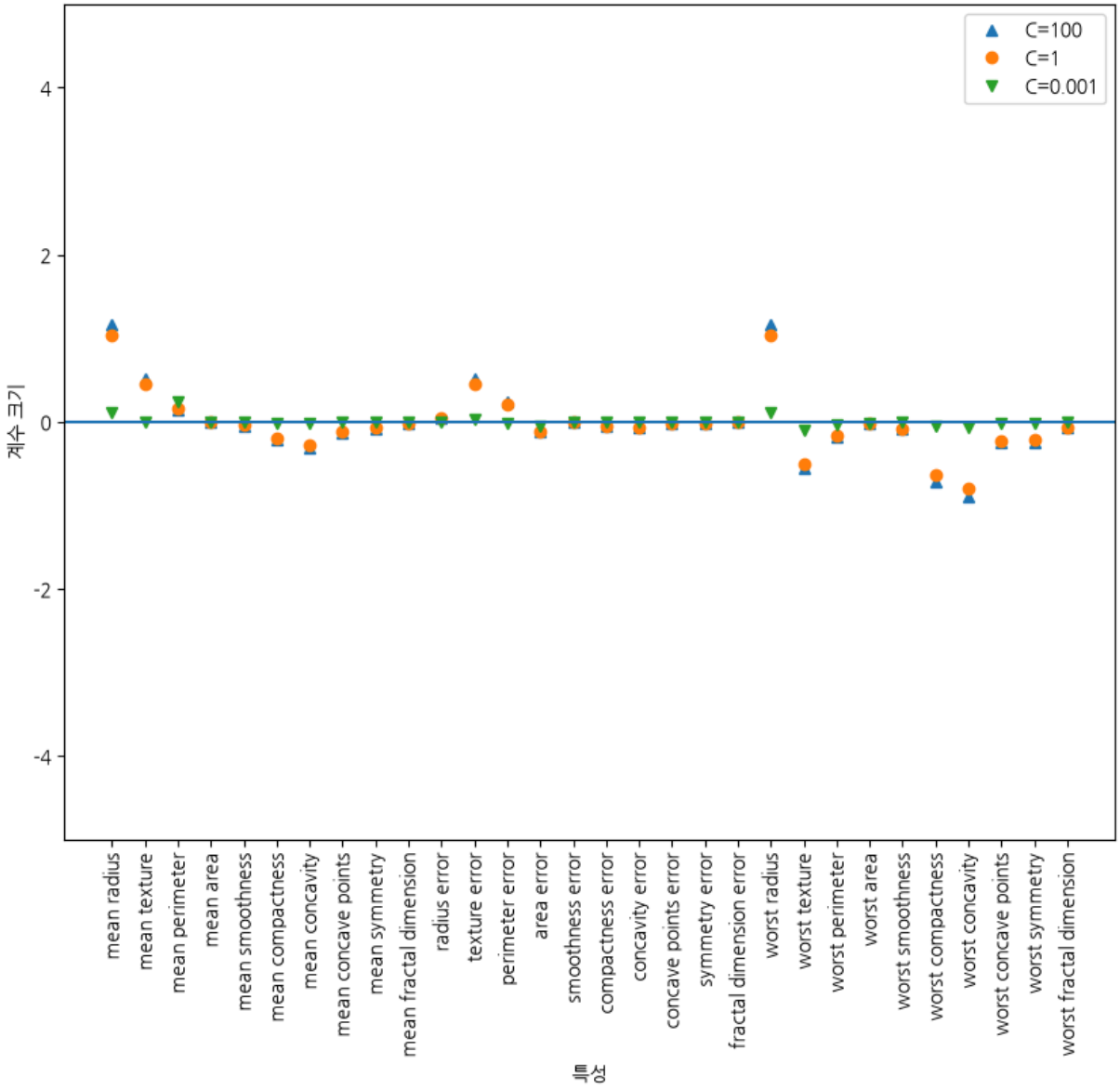
Out[12]: <matplotlib.legend.Legend at 0x1174b8170>



- C의 값이 **클수록 규제를 풀고**, C의 값이 **적을수록 많은 규제**가 이루어진다.
- 로지스틱 회귀(LogisticRegression)은 기본으로 L2규제를 적용하고 있음. (0에 가깝게, 0으로 만들지 않음.)

In [13]:
```
plt.figure(figsize=(10,8))
plt.plot(logreg100.coef_.T, '^', label="C=100")
plt.plot(logreg.coef_.T, 'o', label="C=1")
plt.plot(logreg001.coef_.T, 'v', label="C=0.001")
plt.xticks(range(cancer.data.shape[1]), cancer.feature_names, rotation=90)
xlims = plt.xlim()
plt.hlines(0, xlims[0], xlims[1])
plt.xlim(xlims)
plt.ylim(-5, 5)
plt.xlabel("특성")
plt.ylabel("계수 크기")
plt.legend()
```

Out[13]:
```
<matplotlib.legend.Legend at 0x11755c080>
```



L1규제 적용한 로지스틱 회귀 모델의 계수

- penalty의 매개변수를 이용하여 모든 특성을 사용할지(L2), 일부 특성만 사용할지(L1)을 결정

In [14]:
```
for C, marker in zip([0.001, 1, 100], ['o', '^', 'v']):
    lr_l1 = LogisticRegression(C=C,
                               penalty=None,  # 2026.10 수정: 문자열 'none' 대신
                               solver='lbfgs',
                               max_iter=100).fit(X_train, y_train)
    print("C={:.3f} 인 l1 로지스틱 회귀의 훈련 정확도: {:.2f}".format(
          C, lr_l1.score(X_train, y_train)))
    print("C={:.3f} 인 l1 로지스틱 회귀의 테스트 정확도: {:.2f}".format(
          C, lr_l1.score(X_test, y_test)))
    plt.plot(lr_l1.coef_.T, marker, label="C={:.3f}".format(C))

plt.xticks(range(cancer.data.shape[1]), cancer.feature_names, rotation=90)
xlims = plt.xlim()
plt.hlines(0, xlims[0], xlims[1])
plt.xlim(xlims)
```

```
plt.xlabel("특성")
plt.ylabel("계수 크기")

plt.ylim(-5, 5)
plt.legend(loc=3)
```

```
C=0.001 인 l1 로지스틱 회귀의 훈련 정확도: 0.94
C=0.001 인 l1 로지스틱 회귀의 테스트 정확도: 0.97
C=1.000 인 l1 로지스틱 회귀의 훈련 정확도: 0.94
C=1.000 인 l1 로지스틱 회귀의 테스트 정확도: 0.97
C=100.000 인 l1 로지스틱 회귀의 훈련 정확도: 0.94
C=100.000 인 l1 로지스틱 회귀의 테스트 정확도: 0.97
```

```
.../site-packages/sklearn/linear_model/_logistic.py:1381: FutureWarning: 'pe
nalty' was deprecated in version 1.8 and will be removed in 1.10. To avoid t
his warning, leave 'penalty' set to its default value and use 'l1_ratio' or
'C' instead. Use l1_ratio=0 instead of penalty='l2', l1_ratio=1 instead of p
enalty='l1', l1_ratio set to a float between 0 and 1 instead of penalty='ela
sticnet', and C=np.inf instead of penalty=None.
  warnings.warn(
.../site-packages/sklearn/linear_model/_logistic.py:1421: UserWarning: Setti
ng penalty=None will ignore the C and l1_ratio parameters
  warnings.warn(
.../site-packages/sklearn/linear_model/_logistic.py:599: ConvergenceWarning:
lbfgs failed to converge after 100 iteration(s) (status=1):
STOP: TOTAL NO. OF ITERATIONS REACHED LIMIT

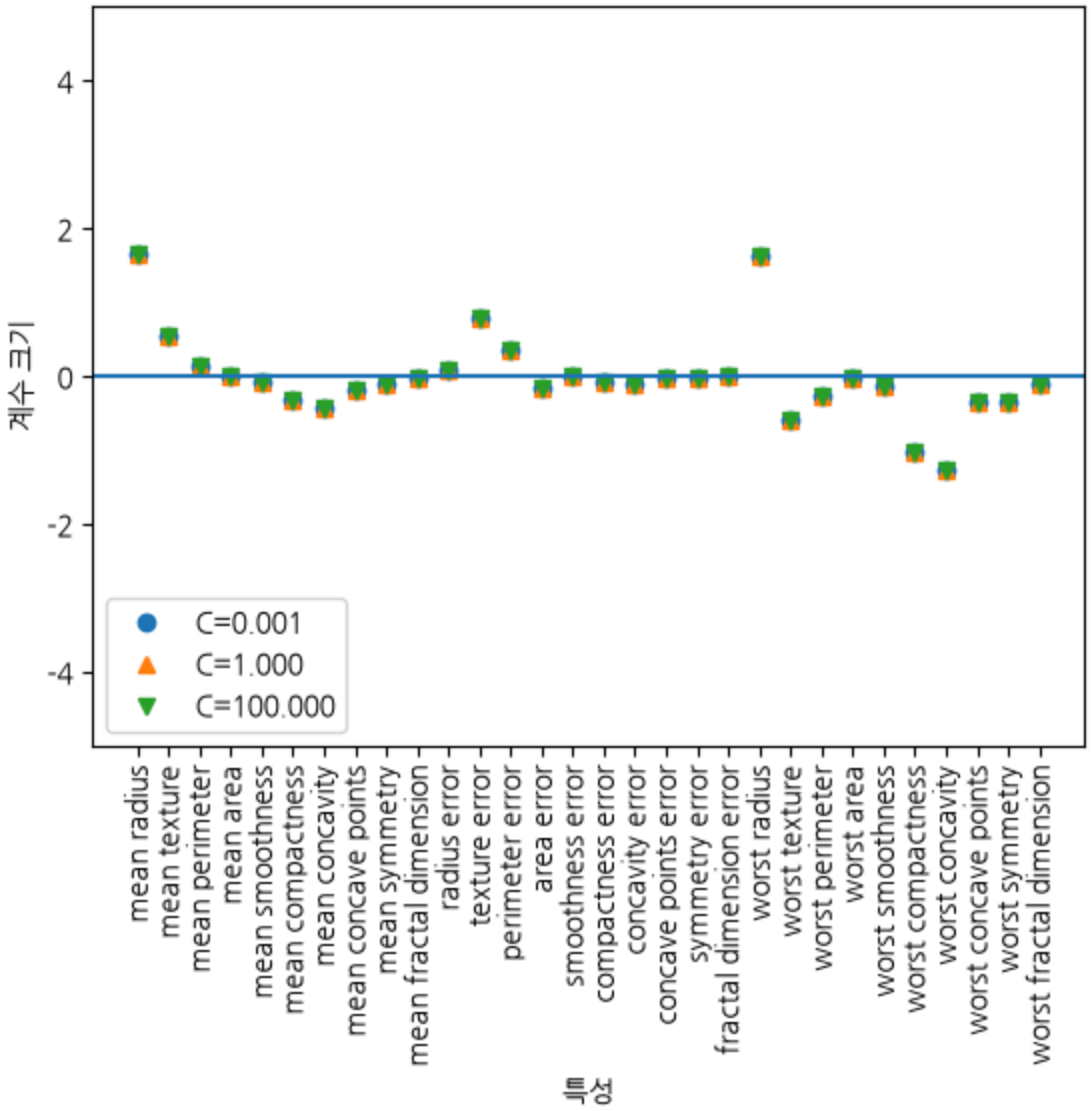
Increase the number of iterations to improve the convergence (max_iter=100).
You might also want to scale the data as shown in:
    https://scikit-learn.org/stable/modules/preprocessing.html
Please also refer to the documentation for alternative solver options:
    https://scikit-learn.org/stable/modules/linear_model.html#logistic-regre
ssion
  n_iter_i = _check_optimize_result(
.../site-packages/sklearn/linear_model/_logistic.py:1381: FutureWarning: 'pe
nalty' was deprecated in version 1.8 and will be removed in 1.10. To avoid t
his warning, leave 'penalty' set to its default value and use 'l1_ratio' or
'C' instead. Use l1_ratio=0 instead of penalty='l2', l1_ratio=1 instead of p
enalty='l1', l1_ratio set to a float between 0 and 1 instead of penalty='ela
sticnet', and C=np.inf instead of penalty=None.
  warnings.warn(
.../site-packages/sklearn/linear_model/_logistic.py:599: ConvergenceWarning:
lbfgs failed to converge after 100 iteration(s) (status=1):
STOP: TOTAL NO. OF ITERATIONS REACHED LIMIT

Increase the number of iterations to improve the convergence (max_iter=100).
You might also want to scale the data as shown in:
    https://scikit-learn.org/stable/modules/preprocessing.html
Please also refer to the documentation for alternative solver options:
    https://scikit-learn.org/stable/modules/linear_model.html#logistic-regre
ssion
  n_iter_i = _check_optimize_result(
.../site-packages/sklearn/linear_model/_logistic.py:1381: FutureWarning: 'pe
nalty' was deprecated in version 1.8 and will be removed in 1.10. To avoid t
his warning, leave 'penalty' set to its default value and use 'l1_ratio' or
'C' instead. Use l1_ratio=0 instead of penalty='l2', l1_ratio=1 instead of p
enalty='l1', l1_ratio set to a float between 0 and 1 instead of penalty='ela
sticnet', and C=np.inf instead of penalty=None.
  warnings.warn(
.../site-packages/sklearn/linear_model/_logistic.py:1421: UserWarning: Setti
ng penalty=None will ignore the C and l1_ratio parameters
  warnings.warn(
.../site-packages/sklearn/linear_model/_logistic.py:599: ConvergenceWarning:
lbfgs failed to converge after 100 iteration(s) (status=1):
STOP: TOTAL NO. OF ITERATIONS REACHED LIMIT

Increase the number of iterations to improve the convergence (max_iter=100).
You might also want to scale the data as shown in:
    https://scikit-learn.org/stable/modules/preprocessing.html
```

```
Please also refer to the documentation for alternative solver options:
    https://scikit-learn.org/stable/modules/linear_model.html#logistic-regre
ssion
  n_iter_i = _check_optimize_result(
```

Out[14]: <matplotlib.legend.Legend at 0x1174b9880>



history

- Machine Learning with sklearn @ DJ,Lim
- date : 2020/12/02 -- version 03

In [ ]: